DEEPSEEK HARNESS 插件 · 微信 / QQ 双端

dsh-qqmusic-plugin

QQ 音乐点歌 + AI 翻唱：让她真的开口唱歌

文档名称：使用手册 & 文件说明

插件版本：v0.9（整理版）

生成日期：2026-10-28

运行环境：Windows 10/11 · DeepSeek Harness · WeChatAgent hook / NapCat OneBot · NVIDIA GPU（8GB 显存即可）

文档内容：插件原理、文件说明、安装配置、微信与 QQ 双端教程、会员登录、排错与测试

目录

1 插件简介
它解决什么问题、能做什么、和现有语音插件的关系

2 工作原理
从一句「唱首晴天」到微信/QQ 里一条完整语音条的全链路

3 环境要求
系统、显卡、Python/RVC 环境、DSH、微信 hook 与 QQ 桥

4 快速开始
五步装好并验证

5 文件说明
每个目录与文件是干什么用的

6 配置说明
profile patch 里的全部配置项

7 使用教程 · 微信端
触发规则、三种模式、自动兜底、找不到歌时

8 使用教程 · QQ 端
给 qq-fish 插件打补丁、触发方式、限制

9 QQ 音乐会员登录
零操作内存提取 / 扫码 / 手动 cookie，以及如何验证

10 运维与排错
日志位置、常见问题表、已修复的坑

11 测试与验证
本地测试、联调脚本

12 安全、隐私与已知限制
cookie 只存本机、版权与使用边界

13 附录 A · 文件速查表
一页纸版本

1 • 插件简介

dsh-qqmusic-plugin 是给 DeepSeek Harness（DSH）里的「她」——微信/QQ 上那个有人格的机器人——加的一个真·唱歌技能。用户在聊天里说一句「唱首晴天」，插件会从 QQ 音乐找歌、截一段副歌， 用 RVC 把原唱的人声换成她自己的 AI 音色，再混回原曲伴奏，最后以**真实语音条**发出来。

能做什么

- **点歌翻唱（cover）**：默认模式，她的音色 + 原曲伴奏，默认 30 秒（8~45 秒可调）。
- **原声片段（original）**：直接放原唱的一小段，2~3 秒出结果，适合只想听一段原曲。
- **清唱（acapella）**：去掉伴奏，只留她的声音。
- **会员音源**：接入 QQ 音乐账号后，会员歌/320k 音源也能用；没有会员则自动用免费档。
- **自动兜底**：长群里模型容易“嘴上说唱、实际不调用工具”，插件会自己接管把唱完。
- **找不到歌时**：自动用她的声音发语音说「我不知道你说的歌怎么唱哦」。

和已有语音插件的关系

微信侧语音条通道（iLink / WeChatAgent hook）和 TTS 发声能力由现有的 `dsh-wechatagent-voice-plugin` 提供；本插件**不重复造轮子**：发语音条复用同一套 本机 hook，只在“找不到歌要道歉”时借用它的 Edge TTS。两者在同一套「扩展工具白名单」里合并， 互不顶掉。

> **一句话理解：**语音插件负责「说话」，本插件负责「唱歌」。唱歌比说话重得多——它依赖 本机 GPU 上的 Demucs + RVC 流水线，第一次点一首歌大约 40~90 秒，之后同一首走缓存会快很多。

2 · 工作原理

一次点歌的完整链路（微信与 QQ 共用前半段，只有最后“怎么发出去”不同）：

> **① 解析请求**　模型调用 `sing_song`，或宿主兜底从聊天文本里抽歌名（如「唱首晴天」→ 晴天）

> **② QQ 音乐搜索**　非官方网页接口搜索；`matchScore` 判断“搜出来的像不像这首歌”，不像就当作没找到

> **③ 解析播放地址**　vkey 接口按会员/免费档依次尝试 M800(320k) → M500(128k) → C400(128k AAC)，下载到本机缓存

> **④ 截取片段**　ffmpeg 取副歌附近 30 秒（优先用 QQ 音乐自己的 30 秒试听窗口，其次取全曲 40% 处）

> **⑤ 分轨（GPU）**　Demucs htdemucs 把人声与伴奏分离成两个 wav

> **⑥ 自动变调**　librosa.pyin 估原唱人声中位音高，按 `12·log2(targetF0/f0)` 算变调半音数

> **⑦ 音色转换（GPU）**　RVC（Applio）把人声换成目标音色（默认 Paimon，可换）

> **⑧ 混音母带**　ffmpeg 把她的声音与原曲伴奏混合，EQ/压缩/loudnorm 处理

> **⑨ 编码**　转 24kHz 单声道 PCM：微信侧再编码成 **SILK** 语音条；QQ 侧同时产出 **MP3** 交给 NapCat 转码

> **⑩ 发送**　微信：本机 WeChatAgent hook `/SendVoiceMsg`；QQ：OneBot `record` 消息

缓存与并发

- 所有中间文件按歌曲 mid 落盘（默认 `D:/…/rvc/songs/qqmusic/<songmid>/`），重复点同一首只重跑变化的部分。
- 同一首歌被多个群/QQ 同时点时会**串行渲染**，后到的直接复用结果——避免两条流水线抢同一个中间文件、产出被截断的短语音（曾经出现过“怎么只有 6 秒”的问题，已修复并有回归测试）。
- 缓存完整性按“请求秒数 × 音频字节率”校验，不完整的文件会被删除重建。

3 • 环境要求

项目	要求
操作系统	Windows 10/11 x64（当前实现依赖 Windows 的本机 hook 与内存读取工具）
DSH	DeepSeek Harness（桌面版），本机已装 profile `desktop`
GPU	NVIDIA 显卡，显存 ≥ 8GB（Demucs + RVC 推理；纯 CPU 会很慢）
RVC 环境	一个可用的 RVC 目录，包含：`venv312`（torch cu128、demucs、soundfile、librosa、faiss）、`applio`（含 core.py、ffmpeg.exe）、`models`（.pth + .index 音色模型）
微信通道	WeChatAgent 本机 hook（默认 `http://127.0.0.1:30001`）及其工作区（会话映射、token）
QQ 通道（可选）	qq-fish 插件（`dsh-xingzhudafeiyu-home`）+ NapCat OneBot（默认 `ws://127.0.0.1:3001`）
QQ 音乐账号（可选）	登录过 QQ 音乐的 Windows 客户端，或扫码/粘贴 cookie；没有账号也能用免费档

> **为什么需要 RVC 环境：** “她的声音”来自 RVC 音色转换，模型是外部资产，插件不负责训练。 换音色只需在配置里改 `model` / `models`（内置 Paimon / DaFeiYu / Ying 三个位置，路径可按需覆写）。

4 • 快速开始

1. **摆放插件源码**：把本文件夹放到任意位置（例如 `<你的目录>/dsh-qqmusic-plugin`），下面的命令都在这个目录里执行。
2. **安装依赖（仅一个）**：

   唯一运行依赖是 `silk-wasm`（微信 SILK 语音条编码，WASM 实现，无需编译）。

   唯一运行依赖是 `silk-wasm`（微信 SILK 语音条编码，WASM 实现，无需编译）。
3. **装进 DSH profile**：

   ```
   node tools/install.mjs
   ```

   它会自动探测本机的 RVC 目录与 WeChatAgent 工作区；探测不到时可用 `--rvc-root <path>` / `--workspace <path>` 显式指定， 也可以指定 `--profile`。 会把插件复制到 `<DSH_HOME>/dsh-qqmusic-plugin`，并往 `profiles/desktop/cordis.patch.yml` 写一段带标记的 insert 配置（同时自动备份 patch 文件）。
4. **重启 DeepSeek Harness**：host 插件只在启动时加载；重启后日志 `~/.dsh/qqmusic/qqmusic.log` 会出现 `apply: registered sing_song, qq_music_search`。
5. **验证**：微信里对她说「唱首稻香」。首次约 40~90 秒，之后同一首很快。 也可以不经过聊天直接联调：

   ```
   node tools/live-sing.mjs --keyword "稻香" --mode original --seconds 8
   ```

> **只想先看效果?** 用 `--mode original` 最快（2~3 秒出结果，不跑 GPU 分轨与音色转换）； `--send` 可直发一条到微信聊天，验证整条发送链路。

5 • 文件说明

共 36 个文件（纯源码 + 文档，不含 `node_modules` 与运行期缓存）。

根目录

文件	用途
`index.mjs`	Cordis 插件入口：解析配置、注册 `sing_song` / `qq_music_search` 两个工具、注入「唱歌规则」系统提示、 给微信人格会话合并工具白名单、挂 `session/event` 自动兜底。 ⚠️ 默认路径含本机目录
`package.json`	包名、版本、`type: module`、运行依赖 `silk-wasm`、DSH bundle patch 声明
`README.md`	安装、配置项表格、工具参数、行为说明、限制（微信 + QQ 两套）
`FILES.md`	文件清单详解（本手册第 5 章的完整版）
`LICENSE / SECURITY.md / .gitignore`	MIT 许可证 / 数据流向与漏洞上报 / 忽略运行期产物
`cordis.patch.yml`	bundle 安装方式的 insert 模板（与 install.mjs 二选一）
`docs/manual.pdf / docs/manual.html`	本手册（18 页 A4）及其 HTML 源文件

src/ — 运行时核心

文件	用途
`qqmusic.mjs`	QQ 音乐非官方接口：搜索、vkey 解析（320k/128k/m4a 多档）、下载、歌词；`matchScore` 歌名相似度防止“搜到什么唱什么”； `rankCandidates` 在有会员时优先原唱/母带版本
`qqauth.mjs`	会员登录态：读取 `~/.dsh/qqmusic/cookie.json`，生成请求用 Cookie 头；支持完整 cookie 原文
`singing.mjs`	**翻唱流水线**：搜索→下载→截段→Demucs→RVC→混音→24k PCM→SILK/MP3； 含同曲渲染互斥锁、按真实时长校验缓存、时长不足自动重渲（核心文件，逻辑最重）
`media.mjs`	ffmpeg / Python 子进程封装（带超时与错误摘要）： 截取、混音、转 24k PCM、PCM→MP3、时长探测
`voice.mjs`	PCM → 微信 SILK 语音条编码（silk-wasm）
`wechat.mjs`	DSH 会话 id → 微信聊天 → wxid 映射，调本机 hook 发原生语音条
`voice-fallback.mjs`	找不到歌时借用 `dsh-wechatagent-voice-plugin` 的 Edge TTS 说「我不知道你说的歌怎么唱哦」 ⚠️ 依赖另一个本地插件
`autosing.mjs`	微信端自动兜底：消息含「唱+歌名」而模型没调用工具时，宿主自己接管； 含歌名抽取、每会话 90 秒节流、非歌名短语过滤
`log.mjs`	追加式日志（`~/.dsh/qqmusic/qqmusic.log`）与错误信息提取

python/ — 交给 RVC 虚拟环境执行

文件	用途
`separate.py`	Demucs `htdemucs` 分出人声/伴奏；输出同目录 `.vocals.wav` / `.accompaniment.wav`；GPU 加速
`convert.py`	调 Applio `core.py infer` 做 RVC 音色转换（f0=rmvpe、contentvec）
`f0.py`	`librosa.pyin` 估原唱中位音高，供自动变调使用

tools/ — 安装与运维（不参与运行时）

文件	用途
`install.mjs`	复制插件到 `<DSH_HOME>` 并写 profile patch；支持 `--remove` 卸载
`lift-cookie.mjs`	从正在运行的 QQ 音乐 Windows 客户端**内存**里提取登录态（零操作），写 `cookie.json`
`qq-lift-cookie.ps1`	上面的 PowerShell 实现（全 ASCII 输出，规避 Windows 脚本编码问题）
`QqMemScan.cs`	内存扫描的 C# 辅助类（P/Invoke `VirtualQueryEx` / `ReadProcessMemory`）
`qq-login.mjs`	QQ 互联扫码登录 → 换取 `qqmusic_key`（QQ 账号路径，备用）
`qq-status.mjs`	用给定 cookie 实际解析并下载一首会员歌，判断账号有没有会员；同时打印会员接口原始数据
`live-sing.mjs`	绕过 DSH 直接跑一次渲染；`--send` 可直发语音条；`--mode` / `--seconds` 可调
`patch-qqfish.mjs`	给 QQ 侧 `dsh-xingzhudafeiyu-home` 插件插入「点歌快速路径」； 自动备份，支持 `--check` / `--remove`
`qqfish-sing.mjs`	补丁写入 qq-fish 的 `lib/sing.mjs` 模板：懒加载本插件渲染，产出可发送的 mp3

test/ — 本地测试

文件	覆盖内容
`unit.test.mjs`	歌名匹配/排序/选段/配置解析（`QQMUSIC_NETWORK_TEST=1` 加联网搜索）
`apply.test.mjs`	用假 ctx 调 `apply()`：工具注册、hook、策略合并、规则注入
`autosing.test.mjs`	歌名抽取 + 自动兜底状态机 + 节流
`notfound.test.mjs`	「找不到歌 → 语音道歉」端到端（`--send` 真发）
`concurrent-render.test.mjs`	并发点同一首歌不再产生截断语音（回归测试）

6 • 配置说明

配置写在 profile patch（`~/.dsh/profiles/desktop/cordis.patch.yml`）的 insert 条目里， 修改后需重启 DSH 生效。常用项如下（完整默认值见 `index.mjs` 顶部）：

字段	默认	说明
`enabled`	true	关掉后不注册工具
`promptEnabled`	true	是否注入「唱歌规则」系统提示
`rvcRoot`	<RVC_ROOT>（自动探测，可配置）	RVC 根目录（models / applio / venv312 都在下面）⚠️
`python / applioDir / ffmpeg`	由 rvcRoot 推导	RVC 的 Python、Applio 目录、ffmpeg 路径，一般不用改
`model`	Paimon	音色模型名（内置 Paimon / DaFeiYu / Ying）
`models`	三个内置项	`{ 名称: { pth, index, targetF0 } }`，可覆写或新增
`pitch / autoPitch`	9 / true	变调半音；autoPitch 打开时按原唱音高自动计算，pitch 仅作兜底
`indexRate / protect / timbre`	0.45 / 0.5 / 1.0	RVC 参数：检索比例、清辅音保护、共振峰
`accompanimentGain`	0.45	混音时伴奏相对人声的音量
`defaultSeconds / maxSeconds`	30 / 45	默认与最长片段（微信语音条上限 60 秒，留余量）
`workDir`	<rvcRoot>/songs/qqmusic	下载与中间文件缓存，按 songmid 分目录
`workspace / hookUrl`	WeChatAgent 工作区 / 127.0.0.1:30001	微信会话映射、token 与语音发送 hook
`authEnabled / cookieFile`	true / ~/.dsh/qqmusic/cookie.json	是否使用会员登录态
`minMatchScore`	30	歌名匹配低于该分数视为“没找到”（QQ 音乐搜索很模糊，防乱唱）
`autoSingEnabled / autoSingCooldownMs`	true / 90000	微信端模型不调用工具时宿主自动唱；每会话冷却
`voiceFallbackEnabled / notFoundVoiceText`	true / 我不知道你说的歌怎么唱哦	找不到歌时的语音道歉

`fallbackToOriginal`	true	翻唱阶段失败时退化为原声片段，保证有声音

7 • 使用教程 • 微信端

7.1 两个工具

工具	参数	作用
`sing_song`	`keyword`（歌名或“歌手 歌名”）、`mode`（cover/original/acapella）、`seconds`（8~45）、`start`（从第几秒起）	唱一段并发成微信语音条，返回一句话总结（歌名、起始秒、时长、音色、变调、耗时）
`qq_music_search`	`keyword`、`limit`	搜歌返回候选（歌名/歌手/时长/是否免费可播），用于不确定点哪首时

7.2 触发规则（已放宽）

只要消息里同时出现「**唱**」和一个歌名，就算点歌，例如：

- 「唱首起风了」「给我唱一段稻香」「你会唱七里香吗」「来一个晴天听听」「唱个孤勇者」
- 「唱两句」→ 自动给 10~15 秒；「唱一整段 / 多唱点」→ 40~45 秒
- 「放一段×× / 听原唱」→ 原声片段；「清唱」→ 无伴奏

私聊和群聊都支持；群聊里她被 @ 或模型判断值得接话时触发，一轮最多一首、不刷屏。

7.3 自动兜底（为什么需要它）

长期存在的群会话里，模型会从旧历史里养成“群里唱不了/发不了语音”的习惯，即使系统提示要求调用工具， 它也可能只回一句“哼两句”。插件因此在 `session/event` 上兜底： 收到「唱 + 歌名」后，如果这一轮模型没有调用 `sing_song`，**宿主自己把这首歌唱完并发出去。** 每会话 90 秒冷却，提取不到像歌名的词不会乱发。

7.4 找不到歌时

QQ 音乐里确实没有或解析不到时，插件会自动用她的声音（Edge TTS）发一条语音： 「我不知道你说的歌怎么唱哦」；然后你只回一句简短文字即可，不要重复道歉或再发语音。

7.5 发送目标

只会发到**当前 DSH 会话绑定的微信聊天**：会话 id → WeChatAgent 会话映射 → 真实 wxid/chatroom， 再交给本机 hook。会话没有绑定微信时工具会明确报错。

8 • 使用教程 • QQ 端

QQ 上的机器人跑在另一个 DSH 插件里（`dsh-xingzhudafeiyu-home`，状态目录 `~/.dsh/qq-fish`），有自己的一整套回复与语音通道（OneBot record → NapCat 转码）。 它没有对外扩展钩子，所以用“前面插一条点歌快速路径”的方式接入：

8.1 安装补丁

```
node tools/patch-qqfish.mjs --check   # 先看是否已打补丁
node tools/patch-qqfish.mjs           # 打补丁（自动备份 engine.mjs）
node tools/patch-qqfish.mjs --remove  # 需要时还原
```

打完后重启 DSH，日志 `~/.dsh/qq-fish/voice.log` 会出现 `sing start / sing sent / sing failed`。

8.2 行为

- 消息里出现「唱 + 歌名」（私聊或被 @）时，先回一句「好，等我一下，本小姐这就唱给你听～」；
- 随后本机渲染翻唱，把 mp3 以 `record` 语音条发出（NapCat 自己转码成 QQ 语音）；
- 找不到歌 → 用她自己的 TTS 说「我不知道你说的歌怎么唱哦」；
- 任何一步失败 → 自动退回原来的文字回复，不影响正常聊天。

8.3 限制

- 节流沿用 qq-fish 自己的 `voiceMinGapSec`（默认 60 秒/会话），防刷屏；
- 依赖已安装的本插件（`<DSH_HOME>/dsh-qqmusic-plugin`），找不到时自动跳过；
- qq-fish 插件升级/重装会覆盖补丁，重新跑一次 `patch-qqfish.mjs` 即可；
- 补丁按固定锚点插入，脚本会做存在性校验；失败时不会改动原文件。

9 • QQ 音乐会员登录

匿名请求只能听免费歌；登录后会员歌与 320k 音源才可用。登录态保存在 `~/.dsh/qqmusic/cookie.json`，插件每次请求自动带上（改 cookie 不需要重启）。 三种接入方式任选其一：

方式 A • 零操作（推荐）

本机 QQ 音乐 Windows 客户端处于登录状态时，直接从进程内存里提取它的网页会话（uin + qqmusic_key）：

```
node tools/lift-cookie.mjs           # 提取并保存
node tools/lift-cookie.mjs --check   # 只看保存了什么
node tools/qq-status.mjs             # 验证会员音源是否可用
```

方式 B • 扫码登录（QQ 账号）

```
node tools/qq-login.mjs
```

生成二维码（`<插件目录>/.qr/qqmusic-qr.png`，约 2 分钟有效）， 用手机 QQ 扫码并在手机上点「确认登录」，脚本自动换取 `qqmusic_key` 并保存。

方式 C • 手动 cookie

```
node tools/qq-status.mjs --cookie "qqmusic_uin=...;qqmusic_key=..." --save
```

从浏览器登录 `y.qq.com` 后，控制台执行 `document.cookie`，筛出 `qqmusic_uin` 与 `qqmusic_key` 即可。

验证会员归属

`qq-status.mjs` 会用该 cookie 实际解析并下载一首“会员专属”歌曲：

- 成功 → 这个账号可以播放会员/高音质音源 ✅
- 失败 → 账号不是会员，或登录态不完整 ❌

> **小知识：** QQ 音乐里微信登录与 QQ 登录的资产不互通。cookie 里带 `wxopenid / tmeLoginType=2` 表示微信账号，带 QQ 号则是 QQ 账号； 用 `qq-status` 实测“能不能下会员歌”最可靠。

退出登录

```
node tools/qq-login.mjs --clear
```

10 • 运维与排错

10.1 日志位置

日志	内容
`~/.dsh/qqmusic/qqmusic.log`	搜索、下载、分轨、变调、转换、混音、SILK、发送、自动兜底
`~/.dsh/qq-fish/voice.log`	QQ 侧：`sing start / sing sent / sing failed` 与 TTS 记录
`~/.dsh/wechatagent-voice/voice.log`	微信语音插件：提示词注入、语音发送
数据缓存	`<rvcRoot>/songs/qqmusic/<songmid>/`（song.mp3、分段 wav、cover、mix、pcm、mp3）

10.2 常见问题

现象	排查 / 处理
微信里她不唱，只回文字	确认重启过 DSH；看 qqmusic.log 有没有 `autosing: ... request`； 自动兜底触发后会先文字后语音，语音可能晚 30~90 秒到达
语音特别短（如 5~6 秒）	旧版本的并发截断问题，已修复：同曲渲染加锁 + 缓存按真实时长校验 + 时长不足自动重渲。仍出现请贴日志，核对 `sing: silk … 6s` 行
QQ 群里只回“哼两句”	补丁没打或没重启：`patch-qqfish.mjs --check`； 重启后 voice.log 应有 `sing start`
找不到歌 / 会员歌失败	先 `qq-status.mjs` 验证会员；免费档下会员歌会走免费版本或报错； 歌名太模糊时会被相似度阈值拦下（`minMatchScore`）
RVC 音色转换卡住很久	numba 缓存目录不可写时会死循环：插件已通过 `NUMBA_CACHE_DIR` 指向可写目录规避；手动跑脚本时也要设置
发语音 400 / 发送失败	本机 hook 只接受 `<workspace>/state/voice/outgoing/*.silk`；检查 WeChatAgent 是否运行、token 是否可读
“这个会话没有绑定微信聊天”	先在微信里给机器人发一条消息（打开会话窗口），再点歌
第一次点歌特别慢	正常：下载 + GPU 分轨 + RVC 首次约 40~90 秒；同一首第二次走缓存会快得多

10.3 已修复的关键坑

- **并发截断（语音只有 6 秒）**：多个群/QQ 同时点同一首歌时，两条流水线抢同一个中间文件， 后启动的读到了写了一半的 PCM。修复：同曲渲染互斥锁 + 缓存按“秒数×字节率”校验 + 时长不足自动重渲。
- **numba 缓存死循环**：librosa/numba 默认把编译缓存写进 venv 的 `__pycache__`， 在受限环境下不可写会忙等。修复：所有 Python 调用统一设置 `NUMBA_CACHE_DIR` 到可写目录。
- **大群“唱不了”习惯**：模型被群历史带偏，不调用工具。修复：提示词硬规则 + 宿主自动兜底。
- **会员歌优先级**：早期会把免费翻唱版排在原唱前面；修复：有会员时优先原唱/母带。

11 • 测试与验证

```
cd dsh-qqmusic-plugin

# 纯本地测试（不需要 GPU）
node test/unit.test.mjs
node test/apply.test.mjs
node test/autosing.test.mjs

# 联网/GPU 联调
QQMUSIC_NETWORK_TEST=1 node test/unit.test.mjs          # 加联网搜索用例
node test/notfound.test.mjs --send                       # 找不到歌→真发语音道歉
node tools/live-sing.mjs --keyword "稻香" --mode cover --seconds 20
node tools/live-sing.mjs --keyword "起风了" --send --chat wechat:private:<id>
QQMUSIC_SPAWN_INHERIT=1 node test/concurrent-render.test.mjs  # 并发截断回归
```

受限终端下调试：如果当前 shell 无法给子进程创建管道（EPERM），设置 `QQMUSIC_SPAWN_INHERIT=1` 让 ffmpeg/Python 直接继承控制台输出即可。

12 • 安全、隐私与已知限制

隐私

- 会员 cookie 只保存在本机 `~/.dsh/qqmusic/cookie.json`，不上传任何服务器；
- 内存提取工具只读取本机 QQ 音乐客户端进程内存，不写对方程序；建议只在本人机器使用；
- 聊天记录与中间音频都在本机缓存目录，可随时整目录删除。

版权与使用边界

- 音色模型是外部资产：本插件不训练、不分发模型，请使用你有权使用的音色；
- 翻唱作品仅供个人学习娱乐，不要用于冒充他人、诈骗、伪造或商业侵权；
- QQ 音乐接口是非官方网页接口，仅用于本机个人使用，接口稳定性和合规性自行把握。

已知限制

- Windows 专属：内存提取、WeChatAgent hook、NapCat 桥都依赖本机环境；
- 需要 NVIDIA GPU（8GB 显存级）与可用的 RVC 环境，纯 CPU 会很慢；
- 微信语音条最长 60 秒，默认 30 秒、上限 45 秒；
- 会员歌受版权/专辑购买限制，个别歌即使有会员也解析不到；
- QQ 侧补丁依赖 `dsh-xingzhudafeiyu-home` 当前版本的文件结构，升级后需重新打补丁。

通用版（本仓库）已做的处理

- **路径自动化**：`rvcRoot`、WeChatAgent 工作区、语音插件目录、cookie 位置全部自动探测， 也都能在配置里覆盖；探测不到只告警，不阻止插件挂载。
- **无个人数据**：没有写死账号、群号、聊天号或本机路径；示例聊天号用占位符。
- **仓库规范**：`LICENSE`（MIT）、`SECURITY.md`、`.gitignore`、 package 元数据与发布前检查清单（见 `FILES.md`）。
- **QQ 补丁更稳**：目标插件版本校验，锚点不匹配时拒绝写入（原文件保持不动），自动备份与 `--remove`。
- **Windows 专属能力**：内存提取、微信 hook 在代码与文档中明确标注；非 Windows 下对应命令直接给出提示， 核心翻唱流水线本身与平台无关。

附录 A • 文件速查表

路径	一句话作用
index.mjs	插件入口：注册工具、提示词、策略、自动兜底
package.json	包声明 + silk-wasm 依赖
README.md / FILES.md	说明文档 / 文件清单
src/qqmusic.mjs	QQ 音乐搜索/取地址/下载/歌词
src/qqauth.mjs	会员登录态读写
src/singing.mjs	翻唱流水线（核心）
src/media.mjs	ffmpeg/Python 封装
src/voice.mjs	PCM→微信 SILK
src/wechat.mjs	会话→wxid→hook 发语音
src/voice-fallback.mjs	找不到歌时借 TTS 道歉
src/autosing.mjs	微信端自动兜底
src/log.mjs	日志
python/separate.py	人声/伴奏分离
python/convert.py	RVC 音色转换
python/f0.py	音高检测（自动变调）
tools/install.mjs	安装进 DSH profile
tools/lift-cookie.mjs	从客户端内存提取会员登录态
tools/qq-lift-cookie.ps1	内存扫描 PowerShell 实现
tools/QqMemScan.cs	内存扫描 C# 辅助
tools/qq-login.mjs	扫码登录（QQ 账号）
tools/qq-status.mjs	验证会员音源
tools/live-sing.mjs	联调脚本（可直发）
tools/patch-qqfish.mjs	给 QQ 端插件打点歌补丁
tools/qqfish-sing.mjs	QQ 补丁的 `lib/sing.mjs` 模板
test/unit.test.mjs	纯函数单测
test/apply.test.mjs	插件挂载测试
test/autosing.test.mjs	自动兜底状态机
test/notfound.test.mjs	语音道歉链路

`test/concurrent-render.test.mjs`	并发截断回归